Name :-Akshat Chauhan

CU24250136

BTech CSE 3rd Year SECTION - "c"

Roll No. :- 05

==========

Experiment No. 1

Experiment Name

Implementation of Fundamental Image Processing Operations

using Python and OpenCV

==========

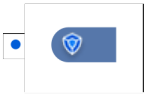
Aim:

To develop a comprehensive understanding of fundamental image

processing techniques by implementing image acquisition,

storage, color space conversion, geometric transformations,

and image enhancement operations using Python and OpenCV.



Step 1: Import Required Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tkinter import Tk, filedialog
import os
```

Step 2: Upload / Select Image from Computer

```
from google.colab import files
import io

print("Please upload an image file (e.g., .jpg, .png, .bmp, .tif).")
uploaded = files.upload()

if not uploaded:
    print("No file uploaded.")
    exit()

image_filename = next(iter(uploaded)) # Get the name of the first uploaded file
image_path = io.BytesIO(uploaded[image_filename])

# Load color image
# For images uploaded via files.upload(), cv2.imread needs a file path or a numpy array.
# We will use cv2.imdecode to read the image from bytes.
image = cv2.imdecode(np.frombuffer(uploaded[image_filename], np.uint8), cv2.IMREAD_COLOR)

if image is None:
    print("Error: Unable to load the selected image.")
    exit()

print("Image loaded successfully.")
print("Selected Image:", image_filename)
```

```
Please upload an image file (e.g., .jpg, .png, .bmp, .tif).
Choose Files No file chosen  Uploadwidgetisonlyavailablewhenthecellhasbeenexecuted in the current browser session. Please rerun this cell to enable.
Saving Screenshot 2025-02-26 132501.png to Screenshot 2025-02-26 132501.png
Image loaded successfully.
Selected Image: Screenshot 2025-02-26 132501.png
```

Step 3: Display Image using OpenCV and Matplotlib

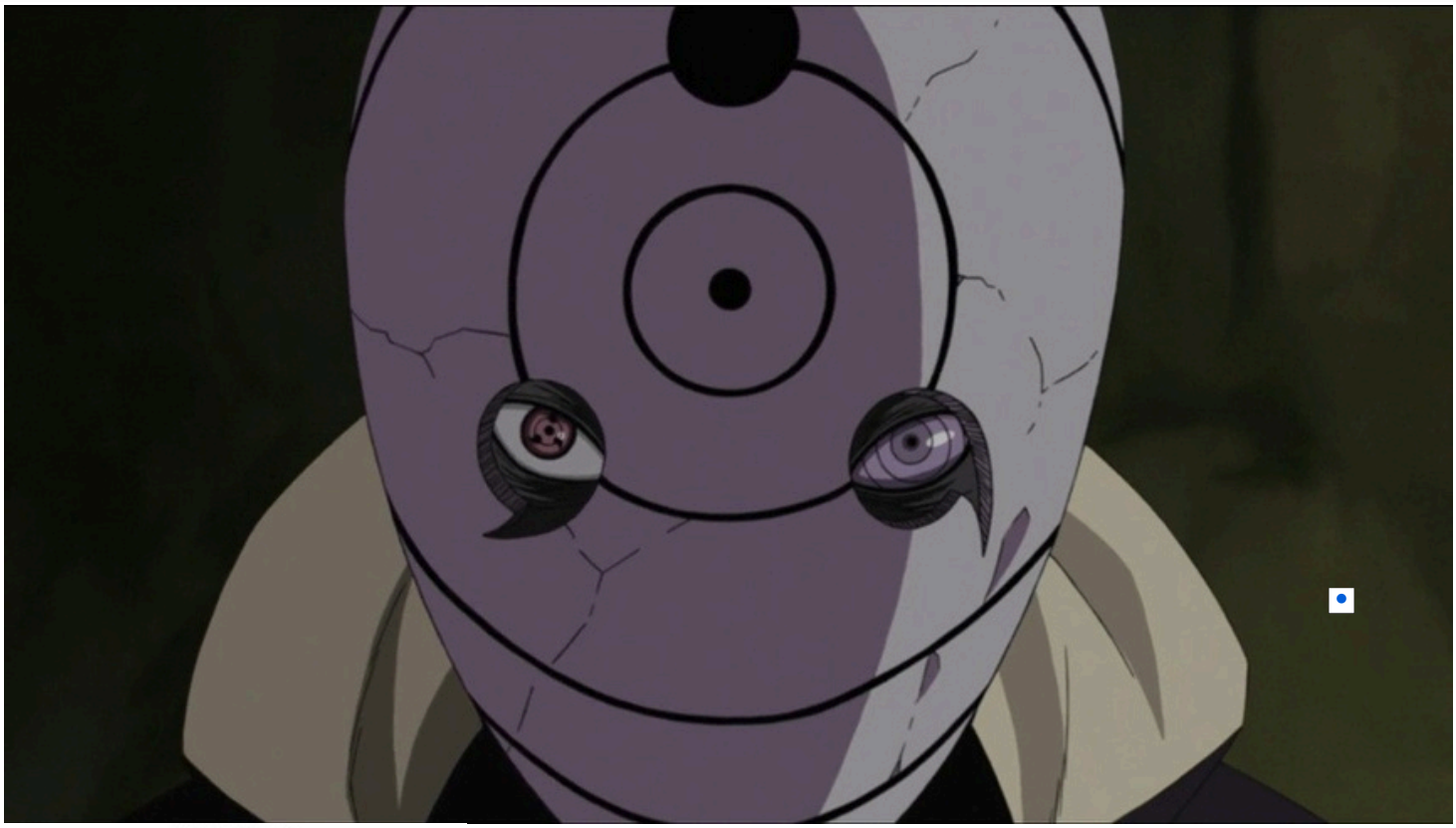
```
from google.colab.patches import cv2_imshow

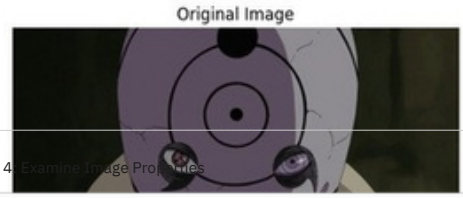
cv2_imshow(image)

# Convert BGR to RGB for Matplotlib
image_rgb = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2RGB
)

  # Display using Matplotlib
plt.figure(figsize=(6,   5))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()

print("Original image displayed successfully.")
```





Step 4: Examine Image Properties

```
height, width, channels = image.shape

print("\n========== IMAGE PROPERTIES ==========")

print("Image Width:", width)
print("Image Height:", height)
print("Image  Resolution:",  width,  "x",  height)
print("Number of Channels:", channels)
print("Image Data Type:", image.dtype)
print("Total Number of Pixels:", height * width)
```

```
========== IMAGE PROPERTIES ==========
Image Width: 1914
Image Height: 1079
Image Resolution: 1914 x 1079
Number of Channels: 3
Image Data Type: uint8
Total Number of Pixels: 2065206
```

Step 5: Save Image in JPEG and PNG Formats

```
jpeg_path = "output_image.jpg"
png_path = "output_image.png"

cv2.imwrite(
    jpeg_path,
    image,
    [cv2.IMWRITE_JPEG_QUALITY, 95]
)

cv2.imwrite(
    png_path,
    image
)

print("\n========== IMAGE STORAGE ==========")

print("JPEG Image Saved:", jpeg_path)
print("PNG Image Saved:", png_path)

print("JPEG File Size:",
      round(os.path.getsize(jpeg_path) / 1024, 2),
      "KB")

print("PNG File Size:",
      round(os.path.getsize(png_path) / 1024, 2),
      "KB")
```

```
========== IMAGE STORAGE ==========
JPEG Image Saved: output_image.jpg
PNG Image Saved: output_image.png
```

```
JPEG File Size: 198.01 KB
PNG File Size: 1131.16 KB
```

Step 6: Convert Image to Different Color Spaces

```
gray_image = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2GRAY
)

# HSV
hsv_image = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2HSV
)

# LAB
lab_image = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2LAB
)

print("\n========== COLOR SPACE CONVERSION ==========")

 print("BGR to Grayscale: Completed")
print("BGR   to   HSV:    Completed")
print("BGR to LAB: Completed")
```

```
========== COLOR SPACE CONVERSION ==========
BGR to Grayscale: Completed
BGR to HSV: Completed
BGR to LAB: Completed
```

Step 7: Resize Image

```
new_width = 400
new_height = 300

resized_image = cv2.resize(
    image,
    (new_width, new_height)
)

print("\n========== GEOMETRIC TRANSFORMATIONS ==========")

print("Image Resizing: Completed")
print("New Resolution:", new_width, "x", new_height)
```

```
========== GEOMETRIC TRANSFORMATIONS ==========
Image Resizing: Completed
New Resolution: 400 x 300
```

Step 8: Rotate Image

```
rotated_image = cv2.rotate(
    image,
    cv2.ROTATE_90_CLOCKWISE
)

print("Image Rotation by 90 Degrees: Completed")
```

```
Image Rotation by 90 Degrees: Completed
```

Step 9: Horizontal and Vertical Flipping

```
horizontal_flip = cv2.flip(
    image,
    1
)

vertical_flip = cv2.flip(
    image,
    0
)

print("Horizontal Flipping: Completed")
print("Vertical Flipping: Completed")
```

```
Horizontal Flipping: Completed
Vertical Flipping: Completed
```

Step 10: Generate Complement / Negative Image

```
negative_image = cv2.bitwise_not(
    image
)

print("\n========== IMAGE COMPLEMENT ==========")
print("Negative Image Generated Successfully.")
```

```
========== IMAGE COMPLEMENT ==========
Negative Image Generated Successfully.
```

Step 11: Crop Region of Interest (ROI)

```
roi_width = width // 2
roi_height = height // 2

x1 = width // 4
y1 = height // 4

x2 = x1 + roi_width
y2 = y1 + roi_height

# Make sure coordinates stay inside the image
x2 = min(x2, width)
y2 = min(y2, height)

roi = image[y1:y2, x1:x2]

print("\n========== REGION OF INTEREST ==========")

print("ROI Coordinates:")
print("Starting Point:", (x1, y1))
print("Ending Point:", (x2, y2))
print("ROI Shape:", roi.shape)

print("ROI Cropping: Completed")
```

```
========== REGION OF INTEREST ==========
ROI Coordinates:
Starting Point: (478, 269)
Ending Point: (1435, 808)
ROI Shape: (539, 957, 3)
ROI Cropping: Completed
```

Step 12: Display Processed Images

```
plt.figure(figsize=(14, 10))

plt.subplot(3, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(3, 3, 2)
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(3, 3, 3)
plt.imshow(
    cv2.cvtColor(hsv_image, cv2.COLOR_HSV2RGB)
)
plt.title("HSV")
plt.axis("off")

plt.subplot(3, 3, 4)
plt.imshow(
    cv2.cvtColor(lab_image, cv2.COLOR_LAB2RGB)
)
plt.title("LAB")
plt.axis("off")

plt.subplot(3, 3, 5)
plt.imshow(
    cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB)
)
plt.title("Resized  Image")
plt.axis("off")

plt.subplot(3, 3, 6)
plt.imshow(
    cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB)
)
plt.title("Rotated  Image")
plt.axis("off")

plt.subplot(3, 3, 7)
plt.imshow(
    cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB)
)
plt.title("Horizontal  Flip")
plt.axis("off")

plt.subplot(3, 3, 8)
plt.imshow(
    cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB)
)
plt.title("Vertical  Flip")
plt.axis("off")

plt.subplot(3, 3, 9)
plt.imshow(
    cv2.cvtColor(negative_image, cv2.COLOR_BGR2RGB)
)
plt.title("Negative  Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 13: Display ROI

```
plt.figure(figsize=(6, 5))

plt.imshow(
    cv2.cvtColor(roi, cv2.COLOR_BGR2RGB)
)

plt.title("Region of Interest (ROI)")
plt.axis("off")

plt.show()

print("\nAll processed images displayed successfully.")
```




```
All processed images displayed successfully.
```

Step 14: Final Output

```
print("\n========== FUNDAMENTAL IMAGE PROCESSING ==========")

print("Image Loaded")
print("Image Resized")
print("Image Rotated")
print("Horizontal Flipping:
print("Vertical Flipping:



print("\nObservations:")

print("Grayscale conversion reduces the image to a single intensity channel.")
print("HSV separates color information into hue, saturation and value.")
print("LAB represents color using lightness and color-opponent components.")
```

```
print("Resizing changes the image resolution.")
print("Rotation changes the orientation of the image.")
print("Flipping creates a mirror image.")
print("Negative operation inverts pixel intensity values.")
print("ROI extraction focuses processing on a selected image region.")

print("\nExperiment Completed Successfully.")
```

```
========== FUNDAMENTAL IMAGE PROCESSING ==========
Image Acquisition: Completed
Image Display: Completed
Image Properties Analysis: Completed
JPEG Storage: Completed
PNG Storage: Completed
Grayscale Conversion: Completed
HSV Conversion: Completed
LAB Conversion: Completed
Image Resizing: Completed
Image Rotation: Completed
Horizontal Flipping: Completed
Vertical Flipping: Completed
Negative Image Generation: Completed
ROI Extraction: Completed



Observations:
Grayscale conversion reduces the image to a single intensity channel.
HSV separates color information into hue, saturation and value.
LAB represents color using lightness and color-opponent components.
Resizing changes the image resolution.
Rotation changes the orientation of the image.
Flipping creates a mirror image.
Negative operation inverts pixel intensity values.
ROI extraction focuses processing on a selected image region.


Experiment Completed Successfully.
```

EXPERIMENT 1 - ANSWERS TO QUESTIONS

QUESTION 1: What is a digital image? Differentiate between grayscale and color images.

A digital image is a representation of a visual scene in the form of discrete pixels.

Each pixel contains numerical information representing the intensity or color at a particular location.

Grayscale Image:

A grayscale image contains only one channel. Each pixel represents intensity, usually ranging from 0 to 255.

0 → Black 255 → White

Color Image:

A color image contains multiple channels. A common representation is RGB or BGR, containing three channels corresponding to different color components.

Therefore:

Grayscale → One intensity channel. Color → Multiple color channels.

QUESTION 2: Explain the difference between RGB, BGR, HSV, and LAB color spaces.

RGB:

RGB stands for Red, Green and Blue. It represents a color using three channels: Red, Green and Blue.

BGR:

BGR stands for Blue, Green and Red. OpenCV normally reads color images in BGR format rather than RGB format.

HSV:

HSV stands for Hue, Saturation and Value.

Hue represents the type of color. Saturation represents the intensity/purity of the color. Value represents brightness.

HSV is useful for color-based image segmentation.

LAB:

LAB represents color using:

L → Lightness A → Green to Red component B → Blue to Yellow component

LAB is useful for image enhancement and color analysis.

Summary:

RGB → General color representation. BGR → OpenCV's common image representation. HSV → Useful for color detection and segmentation. LAB → Useful for perceptual color processing.

QUESTION 3: What is the purpose of converting an image to grayscale before further processing?

Converting an image to grayscale reduces a three-channel color image into a single intensity channel.

This reduces the amount of data that needs to be processed.

Grayscale images are often easier and faster to process for operations such as:

Edge detection Thresholding Image segmentation Feature extraction Grayscale conversion also simplifies many image processing algorithms because color information is not required for every task.

QUESTION 4: Explain the concept of image complement (negative image) and mention its practical applications.

An image complement or negative image is produced by inverting the intensity values of the pixels.

For an 8-bit image:

Negative Pixel = 255 - Original Pixel

Therefore:

Black becomes White. White becomes Black. Dark pixels become bright. Bright pixels become dark.

Practical applications include:

Medical image visualization. Enhancing certain image details. Image analysis. Photograph and document processing. Highlighting information that is difficult to observe in the original image.

QUESTION 5: Differentiate between image resizing, cropping, and scaling.

Image Resizing:

Resizing changes the dimensions or resolution of an image. For example, changing an image from 1920 × 1080 to 800 × 600.

Cropping:

Cropping removes unwanted portions of an image and keeps only a selected region.

Scaling:

Scaling changes the size of an image by a particular scale factor.

For example:

Scaling by 0.5 → Image becomes half its original size. Scaling by 2.0 → Image becomes twice its original size.

In short:

Resizing → Changes image dimensions. Cropping → Selects/removes a region. Scaling → Changes size according to a scale factor. QUESTION 6: What is a Region of Interest (ROI), and why is it important in computer vision?

Region of Interest (ROI) is a specific selected portion of an image that contains the area of interest.

Instead of processing the complete image, computer vision algorithms can focus only on the selected region.

ROI is important because it:

Reduces the amount of data to process. Improves computational efficiency. Focuses analysis on relevant objects. Helps in object detection. Helps in feature extraction. For example, a face region can be selected from an image before performing facial analysis. QUESTION 7: Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

OpenCV is widely used for computer vision because it provides a large collection of optimized image processing and computer vision functions.

It supports:

Image reading and writing. Image transformations. Color space conversion. Filtering. Edge detection. Object detection. Feature extraction. Video processing. OpenCV also works efficiently with NumPy arrays and supports real-time computer vision applications.

Therefore, OpenCV is a practical and powerful library for computer vision development.

QUESTION 8: Explain how image resolution and pixel intensity influence image quality.

Image Resolution:

Resolution refers to the number of pixels used to represent an image.

Higher resolution generally provides more image detail and allows finer structures to be represented.

Lower resolution contains fewer pixels and may result in loss of details.

Pixel Intensity:

Pixel intensity represents the brightness value of a pixel.

In an 8-bit grayscale image:

0 → Black 255 → White

The distribution of pixel intensities affects brightness, contrast and the visibility of image details.

Therefore, both resolution and pixel intensity influence the visual quality of an image.

QUESTION 9: Mention five real-world applications where basic image preprocessing is an essential step.

Five real-world applications are:

Autonomous Driving: Images from cameras are processed before detecting roads, vehicles, pedestrians and traffic signs.

Healthcare: Medical images are preprocessed to improve image quality before diagnosis and analysis.

Surveillance: Images and video frames are processed before object, person and activity detection.

Robotics: Robots preprocess camera images before performing object recognition and navigation.

Industrial Automation: Images are processed before inspecting manufactured products for defects.

These applications require preprocessing to improve the quality and usefulness of input images.

QUESTION 10: How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Image preprocessing improves the quality and consistency of images before they are provided to computer vision algorithms or deep learning models.

Preprocessing can:

Remove unwanted noise. Standardize image size. Improve contrast and visibility. Convert images into suitable color spaces. Normalize image data. Focus processing on important regions. Reduce irrelevant information. Better-prepared images can make important features easier to extract and can help machine learning and deep learning models learn more useful patterns.

Therefore, preprocessing is an important stage before feature extraction and model training.